TopoGym: Library Overview and Environment Specifications

Jason Carlson

August 2026

Scope. This document specifies the TopoGym environment library in depth: the base benchmark (**GridWorld2D**), its textured variant (**GridWorld2D-Texture**), and its topological variant (**GridWorld2D-Top**, non-trivial global topology via edge identifications). Together the three variants form one benchmark, **TopoGym-v1**: the versioned, pinned registry across all variants under the universal action and observation spaces of Section 1.9 — Plain, Texture, and Top are reporting slices of a single suite, and the version pins the environment list (future families extend to v2 without altering v1 entries). This document is the authority on the environments themselves, and Section 4 documents the library's code interfaces with usage examples.

1 GridWorld2D

A registry of named environments backed by one parametric generator. Everything is generated from a frozen configuration and a seed; nothing is hand-placed. The design goals: ground truth matches the theory's objects exactly, so mechanism predictions and certificates are checkable programmatically; and the interface follows existing gym-library conventions — stable named environments with seed variation and a few modifiable parts — so the benchmark is directly usable by others.

1.1 World model

A `size` × `size` grid of cells, partitioned into free and obstacle cells. The agent occupies one free cell; actions are {up, down, left, right}; moving into an obstacle leaves the agent in place. Standing conventions, enforced at generation time:

- the exterior free space (the component containing the start cell) is 4-connected;
- obstacles are well-composed: no 2×2 window contains exactly a diagonal pair of obstacle cells (no diagonal pinch);
- every door has width one: a free cell with obstacle cells on two opposite sides and free cells on the other two;
- a chamber is an obstacle wall enclosing free interior once its doors are sealed; a decoy is a sealed ring enclosing no reachable free cell — its interior is unvisitable by construction, so persistence against it is never rewarded.

1.2 Registry: named environments (the primary interface)

Following the convention of existing gym libraries, TopoGym's canonical interface is a registry of named, pre-defined environments: `gym.make("TopoGym/{Family}-{size}-v0", seed=`n`)`. Each registry entry is a frozen configuration of the underlying generator; the seed drives layout variation (placement, door positions, shape assignment) within that configuration; and a small set of documented parts remains modifiable as `make` kwargs (`p_slip`, `reward_mode`, and per-family knobs noted below). The benchmark is the registry crossed with the published seed lists.

Family	Description	Sizes / variants
`Dilution-{50,200}`	one chamber, no decoys	—
`Chambers2-{50,100,200,400}`	two chambers, fixed geometry; the world-scaling family	—
`ChamberCount{1,2,4,8}-200`	k separated chambers	kwarg: none
`Decoys{0,1,2,4,8}-50`	one chamber among k_d sealed decoys	kwarg: `decoy_side`
`Shape{Sq,Ci,Tr,St}-50`	area-matched chamber shapes	—
`Nested{1,2,3}-50`	sequentially nested shells	—
`GiveUp{1,2,4}-50`	door behind a dead-end corridor of the given length	—
`Bottleneck{3,6}-100`	tree of rooms joined by width-1 corridors of the given length	kwarg: `rooms`
`Maze-{50,100}`	seeded perfect maze (uniform spanning tree over cells)	kwarg: `braid`

Registry ids are stable across releases; new families extend the registry rather than altering existing entries.

1.3 Generator API (advanced)

Under the registry sits the full parametric generator (`topogym.generation.TopoGenConfig2D`), available to power users and used to define registry entries. Its configuration schema:

- `size` — grid side length; any integer accepted.
- `n_chambers`, `n_decoys`; `chamber_side`, `decoy_side`; `chamber_shape`, `decoy_shape` ({square, circle, triangle, star, mixed}, area-matched at equal `side` so shape is never confounded with size).
- `min_sep` — minimum pairwise Chebyshev wall separation, enforced at placement with an up-front packing feasibility check; configurations claiming the one-entry-per-rollout property must satisfy `min_sep` $> L$ (rollout horizon), recorded per configuration in the manifest.
- `doors_per_chamber` (default 1); `door_kind` (`open` — a visible walkable width-1 doorway, the registry convention — or `bump`, hidden until opened by repeated bumps); door width fixed at 1 and validated; `door_corridor_len` (default 0), a dead-end corridor outside each door controlling entry probability q.
- `style` — `open` (alias `rooms`), `nested` (`nested_depth`, `shell_spacing`), `corridor` (`rooms`, `corridor_len`), or `maze` (`braid`); Section 1.4.
- `n_holes` / `target_b1`, `goal_in_chamber`, `seed`; `p_slip` and `reward_mode` are environment-level knobs.

A configuration serializes to the canonical string `TG-GridWorld2D-S{size}-C{c}-D{d}-cs{n}-ds{n}-sep{n}-shp{`
which is the run-log key and the manifest row; registry ids are aliases for canonical strings. Custom configurations built through this API pass the same validity checks and ground-truth generation as registry entries, so results on them remain comparable in kind, but only registry entries belong to the published benchmark.

1.4 Environment Modes

- `open`: chambers and decoys placed in a free field by seeded rejection sampling subject to `min_sep`. The direct realization of the theory's objects: dilution scales with `size`, discrimination with `decoy_count`, parallel entry requires the separation condition. Generation: sample footprints uniformly, reject on overlap or separation violation, carve walls and doors, validate.
- `nested`: `depth` concentric shells around one innermost chamber, one door per shell, doors angularly offset so entry forces traversing shells in order. The sequential regime where the parallel-chamber separation deliberately fails. Bars are born sequentially: entering shell j first exposes shell $j{+}1$'s cycle. Validity: shell spacing ≥ 2; depth $\times$ spacing bounded by half the side.
- `corridor`: rooms joined by width-1 corridors of parameterized length, the room graph constrained to a tree. The tree constraint is load-bearing: a cyclic room graph would enclose wall blocks and manufacture decoy-like H_1 bars, while a tree keeps free space simply connected — zero environmental homology signal, only transient exploration pockets. The bottleneck regime; corridor length is the bottleneck-depth difficulty axis. Ground truth additionally exposes the room graph and corridor cells.
- `maze`: seeded perfect-maze carving (a uniform spanning tree over cells), the maximal-corridor-density cousin of `corridor` mode — simply connected at `braid` $= 0$. The `braid` parameter opens loops with the given density; each opened loop encloses wall and therefore creates an incidental H_1 class, so braided mazes interpolate between the bottleneck regime and the discrimination regime.

1.5 Validity checking and manifests

The generator validates before building and rejects with a reason: packing feasibility (footprint plus separations fits, by a conservative area bound and then detection of rejection-sampling failure); exterior 4-connectivity after carving; well-composedness; door width; separation claims. The manifest tool emits a CSV of (registry id, canonical configuration, validity, satisfied assumptions) covering the registry — and, for power users, any custom configuration set. The published benchmark is the registry manifest plus fixed seed lists — tuning seeds and evaluation seeds, identical environments, disjoint seeds.

1.6 Ground truth, certification, and instrumentation

Read-only accessors per instance: the wall mask; the chamber list with interiors, perimeters, and door cells; the decoy list; per-cell annotations (chamber and door membership); and exact snapshot save/restore. The certified metadata attached to every instance records the free space's homology, computed from the selected complex backend (Section 1.7) and cross-checked against the analytic expectation for the placed features at generation time. Registration-time unit tests assert the standing conventions on every generated instance.

1.7 Complex backends (all variants)

Every environment exposes a `complex` configuration field selecting the geometric substrate used for free-space homology:

- `cubical` (default) — the glued cubical cell complex of the free cells; homology is computed by GUDHI on the order complex of its face poset (the barycentric subdivision), which is robust to every edge identification of Section 3. This backend is the certification source of truth: the metadata's Betti numbers are computed here and cross-checked against the analytic expectation at generation time.
- `rips` — a Vietoris–Rips complex built by GUDHI on the centers of the free cells at a fixed scale $t \in (\sqrt{2}, 2)$, under the quotient metric induced by the base's identification group (wrap translates and flip glide reflections). Orthogonal and diagonal neighbors connect, cells separated by a width-one wall (ambient distance 2) do not, so obstacles produce classes exactly as in the cubical complex; the backend reports dimensions 0 and 1, and the library's tests assert agreement with the certified cubical numbers on every base.

The backend governs the environment-side homology computations (`free_betti`, `visited_betti`, `observed_betti`) and is recorded in the metadata; certification always comes from the cubical complex.

Determinism. The library guarantees end-to-end determinism up to seeds: (configuration, seed) fixes the generated layout and its certified metadata byte-for-byte — including everything computed through GUDHI, since homology is exact linear algebra over $\mathbb{F}_p$ with no randomness — and (environment, reset seed, action sequence) fixes the episode, including `p_slip` resampling, which draws from the episode's seeded generator. Internal iteration orders are sorted so no result depends on interpreter hash state; a cross-process test asserts that two fresh interpreters produce identical layouts, metadata, and rollouts.

1.8 Reward modes (all variants)

Every benchmark variant (GridWorld2D, -Texture, -Top) supports a `reward_mode` configuration field. Episodes have a pre-determined length — $\lfloor 1.2 \max(W, H) \rfloor$ steps for a $W \times H$ world (60 on a 50-grid, 120 on a 100-grid), knowable from the configuration alone and overridable via `max_steps`; short rollouts are deliberate, pairing with lifetime coverage and teleport resets for multi-episode exploration — and every environment places a goal cell by default (removable with `goal=False` for goal-free exploration studies):

- `sparse` (default) — +1 terminal reward on reaching the goal cell, placed inside a designated chamber (in Texture scenarios, the treasure-chest cell). Placement inside a chamber makes steps-to-first-reward coincide with steps-to-first-entry, so the reward metric stays continuous with the exploration metric.
- `none` — no extrinsic reward; the environment is a pure-exploration benchmark, and event-based metrics (entry times, coverage) carry the evaluation.
- `coverage` — +1 for each cell visited for the first time: a dense pure-exploration reward for methods that require a reward signal.
- `deceptive` — the `sparse` target plus a distractor shaping gradient leading away from the target's door: the reward-deception regime of the novelty-search literature, letting reward deception be measured rather than only structural deception.

Ground truth exposes the reward cells and (for `deceptive`) the shaping field. Reward modes exist for community use, for training reward-consuming baselines, and for deception studies; pure exploration evaluations should run `none` to keep an extra variable out of their results.

1.9 Universal action and observation spaces

Both spaces are identical across all variants (GridWorld2D, -Texture, -Top); an agent written against them runs on every environment in the library unchanged.

Action space. `Discrete(4)`: up, down, left, right, always denoting *screen* directions (up decreases y). Moving into an obstacle leaves the agent in place. With `p_slip` > 0 the executed action is resampled uniformly with that probability. In GridWorld2D-Top, crossing an identified edge applies the identification map to the agent's *position* (coordinate reversal where the identification is flipped) — the action space and the meaning of each action are unchanged: left is still screen-left after any crossing. (The optional egocentric `Discrete(3)` interface instead carries a parallel-transported frame, where a flipped seam genuinely mirrors the agent's view.) Family-specific cell mechanics (hazard cells that reset the episode, wormhole cells that teleport; Section 2) are documented with the family and never alter the action set.

Observation space. The canonical observation is a fixed vector: the agent's integer cell coordinates (x, y) followed by a texture block $t \in [0, 1]^{16}$. Slots 0–3 are reserved library-wide for directional blocker adjacency (obstacle to the left, right, above, below of the current cell); slots 4–15 carry per-environment semantic features documented with each family. The texture block is identically zero in GridWorld2D and GridWorld2D-Top; only GridWorld2D-Texture populates it. Optional observation modes for learned baselines — flattened symbolic grid, or an egocentric window of configurable radius with per-cell texture — are available on every variant. Snapshot save/restore is exact (deep copy), and the environment is deterministic given (configuration, seed, action sequence) when `p_slip` $= 0$.

2 GridWorld2D-Texture

The textured variant (`TG-GridWorld2DTex` prefix): the same generator with a per-cell texture channel — semantic labels such as blocker-adjacent (ice), door, ladder, water, and scenario-specific others — exposed in the observation and recorded in ground truth. Texture is a third signal family: local like curvature, but semantic rather than geometric, and only as trustworthy as the environment's labeling.

Design intent. Texture scenarios are built so the taxonomy's boundary is visible: texture discriminates flagged structure cheaply where the labeling is faithful, and fails exactly where discrimination requires enclosure rather than adjacency. The flavor example: arctic sailing — most holes are ice-cube decoys whose neighbors carry ice-adjacent texture (avoidable on sight); the target is a treasure chest in an enclosed space reachable through a water-only passage; texture flags ice adjacency and doorways (ice on both sides), while whether the passage leads anywhere remains invisible to any local feature. Scenario suites should include cells where texture is absent or misleading, so the boundary is measured rather than assumed.

Texture encoding. Textures populate the universal observation's texture block (Section 1.9): slots 0–3 carry directional blocker adjacency (blocker to the left, right, above, below — any combination), and slots 4–15 carry the scenario's semantic features, documented per environment below.

Canonical environments. Eight named environments form the Texture slice of TopoGym-v1:

- `TopoGym/IceShip`. Arctic ship navigation (the agent is the sailboat, and *hitting ice ends the episode* — collisions hurt the hull). Coastal ice sheets attach to the north and south edges — land masses, not floating obstacles — octagonal sealed bergs float in the open water as decoys, and the treasure sits in a cavity deep inside the large eastern landmass, reachable *only* through a guaranteed narrow width-1 channel threaded across the ice (the guarantee is unit-tested: blocking the channel disconnects the goal). Ice-adjacent water cells carry the directional adjacency flags — the boat's collision-warning system — and open-water cells carry the water feature only. Texture makes berg avoidance cheap on sight; whether the channel leads anywhere remains invisible to any local feature.

- `TopoGym/Ladders`. Platforms, ladders, and bridges; a gem on the top platform is the target. Rooms (platforms), ladders, and bridges each carry their own semantic feature; vertical progress is only possible on ladder cells, and bridges connect platforms across gaps. The textured version of the bottleneck regime: ladders and bridges are the corridors, and their texture advertises them.

- `TopoGym/BankRobber`. Nested rooms with the money in the center room — the textured version of the nested regime. Door cells and hallway cells carry their own features, so the sequential structure is advertised locally; the ordering constraint (which door must come first) remains global.

- `TopoGym/DontFall`. A large world with a large central hole: stepping onto the drop resets the episode (the hazard mechanic). A few dozen much smaller huts surround the hole, and exactly one contains the ruby. Textures distinguish drop-adjacent squares, regular dirt, hut interiors, and hut doorways. The hazard makes local novelty signals actively dangerous — the most novel direction is the drop — while the huts reproduce the discrimination regime at scale.

- `TopoGym/SpaceWarp`. Four chambers and a *field* of wormholes: 30+ paired teleporter cells scattered evenly across the map on a jittered lattice (kwarg `warp_sep` sets the minimum spacing; never on or next to a doorway, so every door remains enterable). Stepping onto a wormhole teleports the agent to its partner (the teleport mechanic). The treasure chamber has no door at all: from outside it is indistinguishable from a sealed decoy, and it is enterable only through one wormhole pair running chamber-interior to chamber-interior. Texture flags a cell as a wormhole but never which one — all wormholes are purple — so wormholes are recognizable on sight while their destinations must be explored. The field is the point: it is constant noise for any gradient-follower, while methods that track local transition structure can model each jump once traversed. Reachability of every chamber under wormhole transitions is guaranteed and unit-tested.

- `TopoGym/ClownChase`. The reward-deception scenario. Three sealed decoy tents cluster on one side of the map with a troupe of clown NPCs (`n_clowns` make-kwarg, default two) random-walking among them, each leashed to its own tent; every agent step that reduces the distance to the *nearest* clown pays a tiny reward (10^{-3}) from one shared budget of 2.0 that spans the agent's lifetime on the layout — the distractors run out after a few thousand rewarding steps

in total. The treasure chamber sits on the opposite side (the construction rejects layouts where the sides are not genuinely opposite). Texture distinguishes doors, floor, room interiors, the treasure cell, and clown proximity (a slot lights up when any clown is within one cell); the clowns are otherwise sensed only through the reward gradient — which is exactly the signal that must be distrusted.

- `TopoGym/EnvironmentalIceShip`. IceShip with seasons and a structured landmass: three cavities — one holding the treasure, two empty — each behind its own guaranteed channel, plus two sealed water pockets (little enclosed lakes, visible but unreachable: the certified $b_0 = 3$ counts them). Each episode is drawn sunny (summer) or snowing (winter), with matching palettes; the water texture's value encodes the season (1.0 sunny, 0.5 cold). Only the *floating bergs* respond to the season: in winter they grow — their water fringe freezes in waves at steps 20 and 40, and being on a cell when it freezes ends the episode — and in summer their rims melt away. Channels and the landmass never change. The certified metadata describes the episode-start geometry; a `season` make-kwarg forces either season for controlled evaluation.
- `TopoGym/SearchRescue`. Search and rescue in a collapsed concrete structure. A person is trapped in the one large intact chamber; around it, 160 rubble blocks on a dense lattice leave only width-1/2 passages — a maze the rescuer must thread, with the start placed far from the chamber. Every block is a small H_1 class of its own (certified $b_1 = 161$); in the agent's own discovery filtration — the archive — rubble loops are born small and die quickly as their blocks are circumnavigated, while the chamber's enclosing class grows large and refuses to die. Persistence, not luck, finds the person. Ten explosive red barrels sit in the passages (flame-marked, fatal to step on; neighbors carry the hazard-adjacent texture); a barrel-free route to the person is guaranteed.

SpaceWarp and local evidence (semantics, not soundness). SpaceWarp is built to separate two statements that coincide everywhere else in the library: "this enclosure admits no local entry" and "this enclosure cannot be entered." The treasure chamber's enclosing class is genuinely permanent in the spatial complex (its wall has no door, so the hugging cycle never bounds even after the interior has been visited), and exhaustive local probing of its wall truthfully reports no entry — yet the chamber is enterable, through a non-local transition. Methods that classify enclosures from local evidence alone will mark it unenterable; methods that account for non-local transitions — e.g. by analyzing the transition graph of experience, where a wormhole edge appears once traversed — can do better. SpaceWarp makes that semantic boundary measurable.

Semantic slot assignments. Slots are assigned library-wide so an agent transfers between scenarios; each scenario uses the subset relevant to it. Cell mechanics (hazard, wormhole) are distinct cell types, visible in every observation mode.

slot	feature	slot	feature
0–3	blocker adjacency (L/R/up/down)	10	drop-adjacent
4	water	11	plain ground
5	platform	12	room interior
6	ladder (vertical corridor)	13	standing on a wormhole
7	bridge (horizontal corridor)	14	clown within one cell
8	doorway	15	standing on the treasure
9	hallway		

Scenario sizes are fixed (IceShip/EnvironmentalIceShip/Ladders/BankRobber/SpaceWarp: 50; DontFall/SearchRescue: 61; ClownChase: 60); layouts vary with the seed under each scenario's structural guarantees.

3 GridWorld2D-Top

The topological variant (`TG-GridWorld2DTop` prefix): non-trivial global topology via edge identifications of the grid's boundary. The `topology` field selects the identification:

Value	Identification	Space	$H_1(\cdot;\mathbb{Z})$	$H_1(\cdot;\mathbb{F}_2)$
`plane`	none (walled boundary)	disk	0	0
`cylinder`	left–right, straight	cylinder	$\mathbb{Z}$	$\mathbb{F}_2$
`mobius`	left–right, flipped	Möbius band	$\mathbb{Z}$	$\mathbb{F}_2$
`torus`	both pairs, straight	torus	$\mathbb{Z}^2$	$\mathbb{F}_2^2$
`klein`	one straight, one flipped	Klein bottle	$\mathbb{Z} \oplus \mathbb{Z}/2$	$\mathbb{F}_2^2$
`rp2`	both pairs, flipped	real projective plane	$\mathbb{Z}/2$	$\mathbb{F}_2$

Movement wraps across identified edges, with coordinate reversal where the identification is flipped; obstacles and chambers can be placed on top of any topology. Note the torsion: over $\mathbb{Z}$ the Klein bottle and $\mathbb{RP}^2$ carry $\mathbb{Z}/2$ classes that integer rank alone misses, while $\mathbb{F}_2$ coefficients see them — one reason the library computes homology over $\mathbb{F}_2$.

Why it matters. These spaces are locally flat everywhere: no local signal — curvature, texture, novelty gradient, count statistics — distinguishes a Möbius band from a plane at any single cell, because every neighborhood is Euclidean. The global structure is visible only to signals that aggregate globally, and the ambient H_1 is non-trivial with no obstacles at all. This is the regime designed to break methods that rely on local exploration gradients, and the cleanest possible separation between local and global signal families.

Canonical layout. The Top environments share one scenario shape: chambers placed near the corners of the large fundamental square, exactly one holding the treasure. The square's sides are the edges of the identification diagram (the fundamental polygon), so the corner regions meet across the identified edges — on the torus all four corners are a single point of the quotient; on the Klein bottle and $\mathbb{RP}^2$ they meet in pairs with orientation reversal. What appears locally as four maximally separated chambers is globally a tight cluster reachable across the edges; an agent that carries only local structure treats the corners as far apart, while the quotient makes them neighbors.

Registry entries. The canonical layout is registered per topology as `TopoGym/Top{Plane, Cylinder, Mobius, Torus, Klein, RP2}-50-v0`: four chambers of side 8 near the corners of the fundamental square, exactly one holding the treasure, start near the center. Certified b_1 equals the ambient rank plus the four chamber classes (plane and $\mathbb{RP}^2$: 4; cylinder, Möbius, torus, Klein: 5). The identification-aware Rips backend (Section 1.7) computes homology on the quotient metric and is tested to agree with the cubical certification on every topology. Detection protocols separating the ambient classes from feature classes are left to evaluation code; the metadata provides both the ambient base facts and the composed totals.

4 The library: code interfaces

TopoGym is a Gymnasium library (`pip install topogym`; MIT). This section documents the key abstractions with usage examples; the repository's `docs/` tree carries per-environment pages and the internals reference.

4.1 The registry (the primary interface)

Importing `topogym` registers every TopoGym-v1 id. The seed selects the layout within the frozen configuration; documented knobs pass through `gym.make`:

```
import gymnasium as gym
import topogym  # registers the TopoGym/* ids

env = gym.make("TopoGym/Decoys4-50-v0", seed=3, p_slip=0.05)
obs, info = env.reset(seed=0)
info["topology"]["betti_z2"]         # [1, 5, 0]  (certified)
info["topology"]["betti_z2_sealed"]  # [2, 5, 0]  (doors count as walls)

from topogym import registry
registry.registry_ids()              # every pinned id
registry.canonical_string(
    registry.get_config("TopoGym/Decoys4-50-v0"), seed=3)
# 'TG-GridWorld2D-S50-C1-D4-cs8-ds8-sep2-shpSq-open-slip0-seed3'
registry.manifest(seed=0)            # validity + assumptions per entry
```

Episodes truncate after the pre-determined $4\max(W, H)$ steps; the sparse goal pays $+1$ by default (Section 1.8). Archive-style teleport resets are opt-in:

```
env = gym.make("TopoGym/Maze-100-v0", seed=1, teleport=True)
env.reset(seed=0)
# ... explore ...
env.reset(options={"teleport": (12, 40)})  # any visited cell
```

4.2 The generator and the fluent spec API

Custom configurations pass the same validity checks and certification as registry entries:

```
from topogym.generation import TopoGenConfig2D, generate_2d

cfg = TopoGenConfig2D(
    base="klein", size=25, n_chambers=1, n_decoys=2, n_holes=0,
    chamber_shape="circle", chamber_side=8, min_sep=3,
    door_kind="open", door_corridor_len=2,
)
layout = generate_2d(cfg, seed=42)  # deterministic + certified
layout.metadata.betti_z2

from topogym.spec import Torus
env = Torus(15).holes(3).chambers(1).compile(seed=7)
```

4.3 Measuring discovery

`ExplorationTracker` turns a rollout into discovery-time persistence (essential bars are the real topology of the explored region; finite bars are transient beliefs), and `StatsRecorder` accumulates per-episode metric rows:

```
from topogym.stats import StatsRecorder
from topogym.tda import ExplorationTracker

env = StatsRecorder(gym.make("TopoGym/Nested3-50-v0", seed=1))
tracker = ExplorationTracker(env)
tracker.reset(seed=0)
# ... run the policy ...
tracker.summary()      # coverage, recovery steps, bar counts
env.episodes           # return, coverage, chamber entry steps, ...
m = env.metrics()      # the standardized metric set (frozen dataclass)
m.success_rate; m.sample_efficiency; m.visitation_entropy
m.mean_regret; m.planning_efficiency   # vs env.shortest_path()
m.steps_to_coverage    # {0.5: step, ..., 1.0: step}
m.steps_to_holes       # {k: step the k-th loop was discovered}
```

Structure accessors ground the metrics: `env.topology.betti_numbers_doors_count_as_walls()` / `_dont_count_as_walls()` return a `BettiNumbers` value object; `env.graph()` exposes the free-cell graph as a `networkx.Graph`; `env.shortest_path()` the optimal baseline; `env.bottlenecks()` the straight-through width-1 passage cells (logged at reset under `TOPOGYM_DEBUG`).

Per-step `info` carries `coverage`, `lifetime_coverage` (across episodes on a layout), `chambers_entered`, `episode_return`, `h0_merges`, and friends; `env.unwrapped.chamber_entry_steps` gives each chamber's first-entry step.

4.4 Playing and rendering

Every environment renders as `rgb_array` (procedural pixel-art tiles), `ansi`, or `human` (a pygame window; `pip install topogym[play]`). The keyboard driver:

```
python scripts/play.py --list
python scripts/play.py TopoGym/SpaceWarp-v0 --seed 2
# arrows move; tab reveals hidden structure; r resets;
# backspace regenerates the layout with the next seed
```

Rendering always dims cells outside the agent's current line of sight, so what the agent can and cannot see is visible at a glance (reveal mode shows the full map undimmed; it exists for documentation).

4.5 Debugging the topology live

Two environment variables turn the simulator into a topology debugger; both are off by default and cost nothing when unset.

The H1 overlay. `TOPOGYM_OVERLAY=1` (alias `OVERLAY_ENABLED=1`) recomputes, on every rendered frame, the H_1 classes of the *observed region* — the region the agent has seen and believes free — and draws each class on the grid:

- the **representative cycle** (yellow): the observed cells that currently witness the class, i.e. the inner boundary of the known region around the enclosed pocket. It is loose when the agent has only encircled from afar and tightens as the wall is hugged;
- the **rim** (green): the free cells directly adjacent to the wall component enclosed by the class. A class *with* a rim encloses real structure and will persist; a yellow cycle with *no* green rim encloses only unexplored free space — a transient belief that dies the moment the pocket is explored. Watching transient cycles appear and collapse while real ones tighten onto their rims is the fastest way to build intuition for the discovery filtration of Section 1.

A legend with the live H_1 count sits in the top-right corner. The overlay uses the library's own machinery (observed-region filtration, movement connectivity, the pinch convention), so what it draws is exactly what `observed_betti()` and the `ExplorationTracker` count.

```
# watch loops be conjectured, confirmed, and refuted as you drive:
TOPOGYM_OVERLAY=1 python scripts/play.py TopoGym/Decoys4-50-v0
```

The step stream. `TOPOGYM_DEBUG=1` streams everything the environment computes to the console: a reset line (layout, seed, certified Betti numbers in both door conventions, horizon, reward mode) and one line per step with position, action, reward, running return, termination flags, coverage, lifetime coverage, observed fraction, known components, H0 merges, chamber entries, doors opened, and live scenario state (season and frozen/melted counts, clown positions and remaining budget, hazard counts). The two variables compose:

```
TOPOGYM_DEBUG=1 TOPOGYM_OVERLAY=1 \
    python scripts/play.py TopoGym/SearchRescue-v0
```